SLE-4: Architecture Decision Record (ADR)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM130

Name: Mohasin Riyaj Nadaf

Division: B

Date: 11 October 2026

PART A: Architecture Decision Record (ADR)

Title: Use of Breadth-First Search (with a Visited Set) as the Solver for Maze Path-Finding, instead of Depth-Limited DFS

Status: Accepted

Date: 11 October 2026

PRN & Name: 25UAM130 – Mohasin Riyaj Nadaf

1. Context

My system is a maze path-finding solver and profiler. It takes a grid maze with walls, one start cell and one goal cell, and returns a path of cells from start to goal. I had to choose which uninformed search strategy should be the main solver, because the choice decides whether the path is the shortest possible and how many nodes and how much memory the search needs. In SLE-2 I profiled BFS and depth-limited DFS (limit 1000) on 20×20, 40×40 and 70×70 mazes, and in SLE-3 I drew the C4 model that keeps the neighbour-generation logic common to both algorithms.

2. Decision

I chose Breadth-First Search, using a FIFO queue as the Frontier and a visited set to avoid repeated cells, as the main solver for the maze. DFS stays in the project only as a comparison baseline. Both share the same neighbour-generation function (get_neighbors) and return the same (path_length, nodes_expanded, path) result, so only the Frontier data structure differs.

3. Alternatives Considered

- **Alternative 1 – Depth-limited DFS (limit 1000):** profiled in SLE-2. Needs only O(m) memory and was often fast, but it never returned the shortest path and its cost depends on neighbour

order.

- **Alternative 2 – A* with Manhattan-distance heuristic:** would expand far fewer nodes on a grid maze and stay optimal, but needs a heuristic module that I have not built yet.
- **Alternative 3 – Iterative-Deepening DFS (IDDFS):** optimal with O(d) memory, but it repeats work at every depth level, so it is slower than BFS on mazes of this size.

4. Rationale / Justification

The decision is based mainly on my SLE-2 measurements (5 runs per case, time.perf_counter, manual node counter):

Test case	Algo	Time (ms)	Nodes expanded	Path length
20×20 maze (best case)	BFS	0.0490	47	11
20×20 maze (best case)	DFS (limit 1000)	0.2609	208	59
40×40 maze (average case)	BFS	0.8138	713	47
40×40 maze (average case)	DFS (limit 1000)	0.6565	372	197
70×70 maze (worst case)	BFS	4.7534	3,502	134
70×70 maze (worst case)	DFS (limit 1000)	0.6070	338	298

- **Optimality:** BFS returned 11, 47 and 134 steps, which are the true shortest paths. DFS returned 59, 197 and 298 steps, which is 5.4×, 4.2× and 2.2× longer than optimal. DFS never found the shortest path even once.
- **Efficiency on the small maze:** on the 20×20 maze BFS needed 47 nodes and 0.0490 ms versus 208 nodes and 0.2609 ms for DFS, so it was about 4.4× fewer nodes and 5× faster.
- **Honest exception:** on the 40×40 and 70×70 mazes DFS was faster (0.6565 ms and 0.6070 ms vs 0.8138 ms and 4.7534 ms) and expanded fewer nodes (372 and 338 vs 713 and 3,502). This happened because its fixed neighbour order (down, then right) points towards the goal corner, not because DFS is better; the path it returned was still 4.2× and 2.2× longer.
- **Profiling evidence:** the cProfile flame graph (Fig. 2 of SLE-2) showed bfs() taking about 78% of the ~612 ms profiled against 20% for dfs(), which matches BFS's larger node count. The cost is

acceptable because a 70×70 maze has only 4,900 cells, so BFS's O(b^d) memory stays small.

- **Fits my architecture:** only the Frontier changes (queue for BFS, stack for DFS). Because both use the same get_neighbors, the SLE-2 comparison was fair and the solver can be swapped later without touching the other containers.

5. Consequences

Positive:

- Always gives the shortest path, so results are predictable and easy to verify.
- Simple to implement, test and explain with the C4 diagrams.
- The modular design lets me add a Heuristic Module and move to A* without a rewrite.

Negative / Trade-offs:

- Memory and time grow as O(b^d); on a very large maze (for example 200×200 with a long path) BFS would run out of memory first.
- Can be much slower than DFS on a maze whose layout suits DFS's neighbour order (4.7534 ms vs 0.6070 ms on 70×70), because it explores every cell blindly with no heuristic.

PART B: Short Summary of My SLE Journey

In SLE-1 I built the maze path-finding agent with BFS and DFS code, an AI contribution log and a README. In SLE-2 I profiled both algorithms with cProfile (flame graph) and time.perf_counter on 20×20, 40×40 and 70×70 mazes. BFS was always optimal (11/47/134 steps), while DFS returned 59/197/298 steps although it was sometimes faster on the larger mazes. In SLE-3 I drew the four C4 levels and kept get_neighbors() shared so the comparison stayed fair. In SLE-4 I turned these findings into one decision: use BFS as the main solver, and treat A* with a heuristic as the next step for larger mazes.

PART C: AI Contribution Note

- **AI tools used:** Claude (Anthropic).
- **What AI helped with:** drafting the ADR layout from the SLE-4 guideline and the sample format, summarising my SLE-2 report into the ADR sections, and formatting this Word file.
- **What I did myself:** wrote and ran the BFS/DFS maze code, chose the maze sizes, collected the SLE-2 numbers, drew the C4 model, chose which decision to record, and checked that every number and component name here matches my own work. I will explain and defend the decision in the viva.